

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview
2. Background & Business Objective
3. Product Purpose
4. System Functionalities
5. User Stories & Use Cases
6. Features Included (Scope Definition)
7. Features Not Included (Scope Control)
8. Assumptions & Constraints
9. Risks & Questions Throughout Development

Project Name: UM CRM

Version: 1.0

1. Project Overview

Problem Statement:

Sales teams and account managers often find it difficult to keep their customer relationship pipelines up to date. Updating CRMs requires manual data entry, constant context-switching between chat applications (WhatsApp/Telegram) and the CRM portal, and tedious administrative work. This friction leads to outdated deal stages, forgotten follow-ups, and ultimately, lost revenue.

Target Domain:

Enabling rapid, zero-friction sales enablement and automated customer relationship management through conversational AI.

Proposed Solution Summary:

UM CRM removes the friction of manual data entry by enabling users (both sales reps and clients) to interact with the CRM through simple, natural language on their preferred chat apps. Powered by a LangGraph multi-agent architecture, the system instantly translates unstructured conversations into structured CRM operations (logging activities, updating deal stages, creating contacts) while providing a high-fidelity visual Kanban dashboard for sales managers. This shifts the sales rep's role from data-entry clerk to strategic closer.

2. Background & Business Objective

Background of the Problem:

For many years, managing a sales pipeline followed a manual, linear pathway. A sales rep chats with a client on WhatsApp, concludes the conversation, opens a separate CRM web portal, navigates to the client's profile, and manually types out a summary of the meeting and updates the deal stage. This process requires extensive labor, discipline, and constant context-switching, which blocks sales teams from focusing on actual selling.

Importance of Solving This Issue:

Accelerating the time to update CRM records from minutes of manual work to seconds of automated AI parsing. By breaking the administrative barrier, sales organizations can maintain a real-time, 100% accurate "Single Source of Truth" without relying on human discipline to manually log every interaction.

Strategic Fit / Impact:

It aligns with the modern push towards Agentic AI workflows. By integrating a sophisticated multi-node AI agent (via OpenRouter/Ilmu AI) directly into communication webhooks, UM CRM facilitates an ecosystem where the CRM manages itself, deeply integrating into the sales team's existing workflow rather than forcing them to adopt a new one.

3. Product Purpose

3.1. Main Goal of the System:

To provide an AI-enabled, autonomous partner in sales and customer relationship management.

3.2. Intended Users (Target Audience): - Sales Representatives - Account Managers - Business Development Professionals - SME Founders / Entrepreneurs - Customers/Clients (interacting via chatbot interfaces)

4. System Functionalities

4.1. Description:

The system operates as a high-speed generative CRM engine through natural language. Using a Prompt-to-Database model, the LangGraph intelligence captures conversational intent, identifies required CRM operations, and updates the database accordingly, reflecting instantly on a visual Next.js dashboard.

4.2. Key Functionalities: - **Conversational CRM Updates:** Translate human-language chat messages (via WhatsApp/Telegram) into structured database inserts and updates. - **Visual Pipeline Management:** A React-based, drag-and-drop Kanban board for managers to visually track opportu-

nity stages in real-time. - **Autonomous Activity Tracking:** Automatically logs calls, emails, and meetings parsed from chat histories. - **Multi-channel Webhooks:** Universal endpoint ingestion supporting multiple external chat providers.

4.3. AI Model & Prompt Design

4.3.1. Model Selection:

The system utilizes OpenRouter and Ilmu AI LLMs. These models were chosen for their high reasoning capabilities, strict JSON output formatting, and ability to accurately execute function calling (tool use), which is critical for mapping natural language directly to exact PostgreSQL database schemas.

4.3.2. Prompting Strategy:

The team utilizes **multi-step agentic prompting** via the LangGraph framework. - **Reasoning:** Instead of relying on a single, massive zero-shot prompt which is prone to hallucination, the system routes requests through a triad-node architecture: 1. *Gatekeeper Node*: Evaluates intent and filters out-of-scope requests. 2. *Manager Node*: Decomposes the approved intent into a sequence of CRM tools. 3. *Worker Node*: Executes the tools and validates the output. This strategy ensures safe, isolated, and deterministic CRM task execution.

4.3.3. Context & Input Handling:

The system manages unstructured inputs by maintaining a stateful **messages** array. To prevent context window overflow, the system truncates the history to the last 5 relevant conversational turns. - **Maximum Input Size:** The web-hook accepts payloads up to approximately 2000 tokens. - **Exceeding Limits:** If an input exceeds this limit, the FastAPI backend chunks the input or gracefully rejects the payload with an HTTP 413 error, informing the user via chat to send a shorter message.

4.3.4. Fallback & Failure Behavior:

When the model returns off-topic responses, the **Gatekeeper Node** safely intercepts and returns a polite refusal ("I can only assist with CRM tasks."). If the LLM hallucinates a non-existent CRM tool or provides invalid parameters, the **Worker Node** catches the **Pydantic** validation error. Instead of crashing, it feeds the error back into the state, prompting the **Manager Node** to ask the user for clarification (graceful error state).

5. User Stories & Use Cases

User Stories: - "As a sales representative, I want the AI to automatically log a meeting and update the deal stage when a client confirms an appointment on WhatsApp, so I don't have to manually enter it into the CRM." - "As a sales manager, I want to visually drag and drop deals across a Kanban board so I can quickly assess my team's revenue forecast." - "As a customer, I want to ask

the business’s WhatsApp number for a project update and receive an instant, context-aware reply without waiting for a human.”

Use Cases (Main Interactions): - **Prompt to CRM Flow:** The customer sends a message -> The Webhook parses the payload -> The LangGraph Gatekeeper identifies intent -> The Manager assigns the `create_activity` task -> The Worker executes the database change -> The Customer receives a confirmation reply. - **Kanban Board Management:** User logs into the Next.js dashboard -> Navigates to Opportunities -> Clicks and drags an opportunity card from “Negotiation” to “Closed Won” -> The React frontend updates the backend API instantly.

6. Features Included (Scope Definition)

- An autonomous AI Customer Service Agent integrated via WhatsApp/Telegram webhooks.
 - A 2D visual Kanban board for drag-and-drop opportunity management.
 - Comprehensive Contact, Opportunity, and Activity tracking directories.
 - Secure JWT-based authentication for the web dashboard.
-

7. Features Not Included (Scope Control)

- Implementation of advanced financial billing, invoicing, or payment gateways.
 - Native mobile application deployments (iOS/Android). The current scope is limited to Responsive Web and Chatbot interfaces.
 - Mass marketing, bulk email campaigns, or deep bi-directional syncing with Google Workspace.
-

8. Assumptions & Constraints

LLM Cost Constraint:

The estimated token cost is roughly \$0.005 per average user session. To keep inference costs manageable at scale, the system implements a “fast-path” regex router in the Gatekeeper node. If a user simply says “Hi” or “Thanks”, the system returns a cached, static response, bypassing the LLM entirely and saving tokens.

Technical Constraints:

The AI agent is currently constrained to text-based interactions. Audio messages (voice notes) or image-based inputs on WhatsApp are not supported in the current MVP.

Performance Constraints:

Because multi-step reasoning requires sequential LLM calls, there is a latency constraint. The webhook must process the graph and return a response within the platform's timeout limits (typically 10-15 seconds), requiring highly optimized async Python functions.

User Input:

The AI model relies heavily on the clarity of the human prompt. Highly ambiguous requests require human-in-the-loop clarification, as the system is strictly constrained from guessing critical database deletions.

9. Risks & Questions Throughout Development

Data Privacy & Cross-Pollination:

How are we ensuring that the AI agent does not accidentally leak data from Client A when responding to Client B? *(Mitigation: Strict RLS-style querying in the Worker node tied to the authenticated webhook session).*

Intent Misinterpretation:

What happens if the AI misunderstands a sarcastic remark and moves a million-dollar deal to "Closed Lost"? *(Mitigation: Implementing rollback features and requiring human confirmation for destructive or high-value state changes).*

Platform Dependency:

How do we maintain stability if the external LLM provider (OpenRouter/Ilmu AI) experiences a major outage? *(Mitigation: Implementing a graceful degradation state where the webhook auto-replies that the system is temporarily offline).*